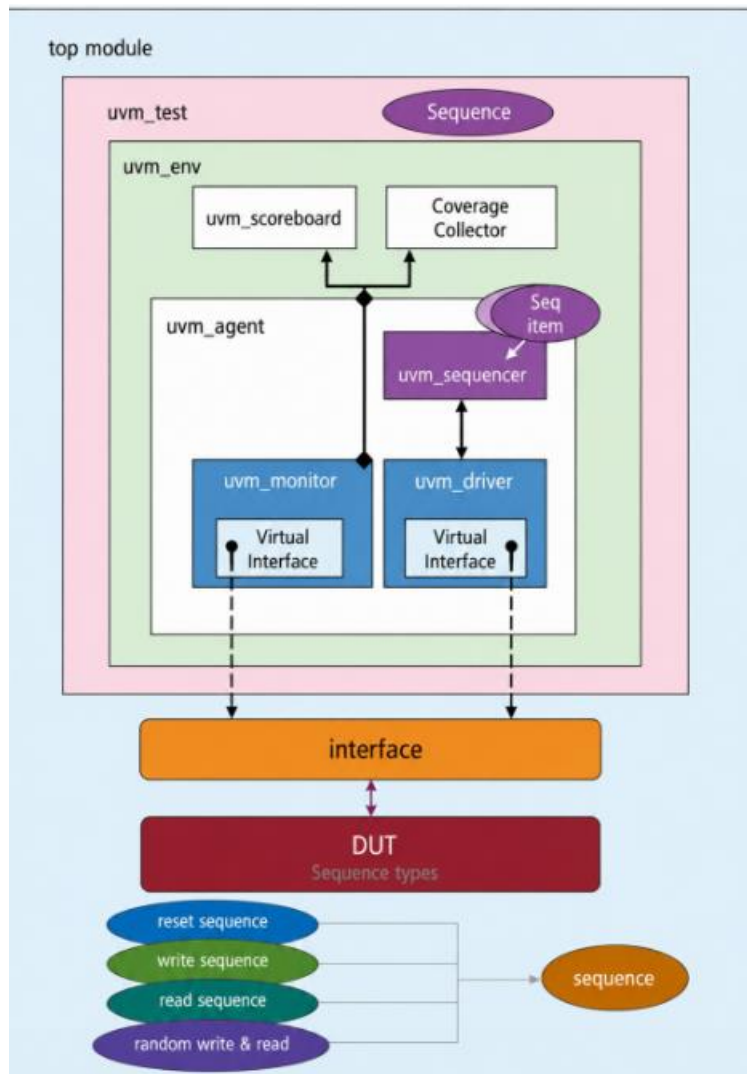


Uvm structure:



How the uvm work:

1. Top Module

The execution begins in the top module, which is responsible for:

- Instantiating the DUT (Design Under Test)
- Instantiating the interface that connects the DUT with the testbench
- Setting the virtual interface using `uvm_config_db`
- Calling `run_test()` to start the UVM environment

This module acts as the bridge between the DUT and the UVM testbench.

2. Test (uvm_test)

The test controls the overall verification scenario:

- It configures the environment
- Selects and starts sequences
- Controls which type of stimulus is applied (e.g., reset, write, read, random)

3. Environment (uvm_env)

The environment contains all verification components:

- **uvm_agent**
- **scoreboard**
- **coverage collector**

It organizes communication and ensures all components work together.

4. Agent (uvm_agent)

The agent is responsible for handling the interface and consists of:

- **Sequencer**
- **Driver**
- **Monitor**

5. Sequence & Sequencer

- The **sequence** generates transaction-level stimuli (sequence items)
- The **sequencer** controls the flow of these transactions and sends them to the driver

6. Driver

The driver converts high-level transactions into pin-level signals:

- Receives sequence items from the sequencer

- Drives signals onto the DUT through the **virtual interface**

7. Interface

The interface connects the testbench to the DUT:

- Contains signals used for communication
- Is accessed using a virtual interface in driver and monitor

8. Monitor

The monitor observes DUT signals:

- Captures input/output activity from the interface
- Converts them into transactions
- Sends them to the scoreboard and coverage collector

9. Scoreboard

The scoreboard checks correctness:

- Compares DUT output with expected results (reference model)
- Reports mismatches as errors

10. Coverage Collector

The coverage component measures verification completeness:

- Tracks which scenarios have been tested
- Ensures all cases are covered

The design:

```
module FIFO(FIFO_if.DUT fifoif);
parameter FIFO_WIDTH = 16;
parameter FIFO_DEPTH = 8;
localparam max_fifo_addr = $clog2(FIFO_DEPTH);
```

```

reg [FIFO_WIDTH-1:0] mem [FIFO_DEPTH-1:0];
reg [max_fifo_addr-1:0] wr_ptr, rd_ptr;
reg [max_fifo_addr:0] count;

always @(posedge fifoif.clk or negedge fifoif.rst_n) begin
    if (!fifoif.rst_n) begin
        wr_ptr <= 0;
        fifoif.wr_ack <= 0; //missing after reset
        fifoif.overflow <= 0; //missing after reset
    end
    else if (fifoif.wr_en && count < FIFO_DEPTH) begin
        mem[wr_ptr] <= fifoif.data_in;
        fifoif.wr_ack <= 1;
        wr_ptr <= wr_ptr + 1;

    end
    else begin
        fifoif.wr_ack <= 0;
        if (fifoif.full && fifoif.wr_en) // & must be &&
            fifoif.overflow <= 1;
        else
            fifoif.overflow <= 0;
    end
end

always @(posedge fifoif.clk or negedge fifoif.rst_n) begin
    if (!fifoif.rst_n) begin
        rd_ptr <= 0;
        fifoif.underflow <= 0; //missing after reset

    end
    else if (fifoif.rd_en && count != 0) begin
        fifoif.data_out <= mem[rd_ptr];
        rd_ptr <= rd_ptr + 1;

    end
    else begin
        if(fifoif.empty && fifoif.rd_en)begin
            fifoif.underflow <= 1;
        end
        else begin
            fifoif.underflow <= 0;
        end
    end
end
end

```

```

always @(posedge fifoif.clk or negedge fifoif.rst_n) begin
    if (!fifoif.rst_n) begin
        count <= 0;
    end
    else begin
        if ( ({fifoif.wr_en, fifoif.rd_en} == 2'b10) && !fifoif.full)
            count <= count + 1;
        else if ( ({fifoif.wr_en, fifoif.rd_en} == 2'b01) && !fifoif.empty)
            count <= count - 1;
        else if ( ({fifoif.wr_en, fifoif.rd_en} == 2'b11) && fifoif.empty)
            //missing case
            count <= count + 1;
        else if ( ({fifoif.wr_en, fifoif.rd_en} == 2'b11) && fifoif.full)
            count <= count - 1;
    end
end

assign fifoif.full = (count == FIFO_DEPTH)? 1 : 0;
assign fifoif.empty = (count == 0)? 1 : 0;
//assign fifoif.underflow = (fifoif.empty && fifoif.rd_en)? 1 : 0;    // its
mentioned in pdf its a sequential

assign fifoif.almostfull = (count == FIFO_DEPTH-1)? 1 : 0;    //bug in -2
assign fifoif.almostempty = (count == 1)? 1 : 0;

endmodule

```

config:

```

package fifo_cfg_pkg;

import uvm_pkg::*;
`include "uvm_macros.svh"

class fifo_config extends uvm_object;
    `uvm_object_utils(fifo_config)

    virtual FIFO_if FIFO_vif;

```

```

function new(string name = "fifo_config");
    super.new(name);
endfunction

```

```

endclass

```

```

endpackage

```

driver:

```

package fifo_driver_pkg;
import uvm_pkg::*;
import fifo_seq_item_pkg::*;
import fifo_cfg_pkg::*;
`include "uvm_macros.svh"

```

```

class fifo_driver extends uvm_driver #(fifo_seq_item);
`uvm_component_utils(fifo_driver)

```

```

virtual FIFO_if FIFO_vif;
fifo_seq_item stim_seq_item ;
fifo_config fifo_cfg ;

```

```

function new(string name="fifo_driver", uvm_component parent=null);
    super.new(name, parent);
endfunction

```

```

function void build_phase (uvm_phase phase ) ;
super.build_phase(phase ) ;
if(!uvm_config_db #(fifo_config):: get(this, "" ,"CFG",fifo_cfg)) begin
`uvm_fatal("build_phase" , "unable to get config")
end
endfunction

```

```

function void connect_phase (uvm_phase phase);
super.connect_phase (phase);

```

```

FIFO_vif = fifo_cfg.FIFO_vif ;
endfunction

```

```

task run_phase(uvm_phase phase);
    super.run_phase(phase);

```

```

    forever begin
        stim_seq_item = fifo_seq_item::type_id::create("fifo_seq_item");

        seq_item_port.get_next_item(stim_seq_item);

        FIFO_vif.data_in    = stim_seq_item.data_in;
        FIFO_vif.rst_n      = stim_seq_item.rst_n;
        FIFO_vif.wr_en      = stim_seq_item.wr_en;
        FIFO_vif.rd_en      = stim_seq_item.rd_en;
        @(negedge FIFO_vif.clk);
        seq_item_port.item_done();

    end
endtask

endclass

endpackage

```

env:

```

package fifo_env_pkg;
import fifo_driver_pkg::*;
import fifo_coverage_pkg::*;
import fifo_scoreboard_pkg::*;
import fifo_agent_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"

class fifo_env extends uvm_component;
`uvm_component_utils(fifo_env)

    fifo_agent agt;
    fifo_scoreboard sb;
    fifo_coverage cov;

    function new (string name = "fifo_env", uvm_component parent = null);

```

```

        super.new(name, parent);
endfunction

function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    agt = fifo_agent::type_id::create("agt", this);
    sb = fifo_scoreboard::type_id::create("sb", this);
    cov = fifo_coverage::type_id::create("cov", this);

endfunction

function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    agt.agt_ap.connect(sb.sb_export);
    agt.agt_ap.connect(cov.cov_export);
endfunction

endclass
endpackage

```

coverage:

```

package fifo_coverage_pkg;

import fifo_seq_item_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"

class fifo_coverage extends uvm_component;
    `uvm_component_utils(fifo_coverage);

    uvm_analysis_export #(fifo_seq_item) cov_export;
    uvm_tlm_analysis_fifo #(fifo_seq_item) cov_fifo;
    fifo_seq_item seq_item_cov;

    //coverage group

    covergroup FIFOcg ;

        wr_en_cp: coverpoint seq_item_cov.wr_en;
        rd_en_cp: coverpoint seq_item_cov.rd_en;

```



```

full_cp: coverpoint seq_item_cov.full;
almostfull_cp: coverpoint seq_item_cov.almostfull;
empty_cp: coverpoint seq_item_cov.empty;
almostempty_cp: coverpoint seq_item_cov.almostempty;
overflow_cp: coverpoint seq_item_cov.overflow;
underflow_cp: coverpoint seq_item_cov.underflow;
wr_ack_cp: coverpoint seq_item_cov.wr_ack;

//ignored when rd_en =1 with full not important
full_cross: cross wr_en_cp, rd_en_cp, full_cp{
    ignore_bins rd_full = binsof(rd_en_cp)intersect{1} &&
binsof(full_cp)intersect{1}
;
}
almostfull_cross: cross wr_en_cp, rd_en_cp, almostfull_cp;
empty_cross: cross wr_en_cp, rd_en_cp, empty_cp;
almostempty_cross: cross wr_en_cp, rd_en_cp, almostempty_cp;

//ignored when wr_en =0 and overflow =1 not important
overflow_cross: cross wr_en_cp, rd_en_cp, overflow_cp{
    ignore_bins not_wr_overflow = binsof(wr_en_cp)intersect{0} &&
binsof(overflow_cp)intersect{1}
;
}

//ignored when rd_en =0 and underflow =1 not important
underflow_cross: cross wr_en_cp, rd_en_cp, underflow_cp{
    ignore_bins not_rd_overflow = binsof(rd_en_cp)intersect{0} &&
binsof(underflow_cp)intersect{1}
;
}

//ignored when wr_en=0 with wr_ack =1 not important
wr_ack_cross: cross wr_en_cp, rd_en_cp, wr_ack_cp{
    ignore_bins not_wren_wrack = binsof(wr_en_cp)intersect{0} &&
binsof(wr_ack_cp)intersect{1}
;
}

endgroup

function new(string name="fifo_coverage", uvm_component parent);
    super.new(name, parent);
    FIFOcg = new();
endfunction

function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    cov_export = new("cov_export", this);

```

```

        cov_fifo = new("cov_fifo", this);
endfunction

function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    cov_export.connect(cov_fifo.analysis_export);
endfunction

task run_phase(uvm_phase phase);
    super.run_phase(phase);
    forever begin
        cov_fifo.get(seq_item_cov);
        FIFOcg.sample();
    end
endtask

endclass

endpackage

```

agent:

```

package fifo_agent_pkg;

import fifo_cfg_pkg::*;
import fifo_driver_pkg::*;
import fifo_sequencer_pkg::*;
import fifo_monitor_pkg::*;
import fifo_seq_item_pkg::*;

import uvm_pkg::*;
`include "uvm_macros.svh"

class fifo_agent extends uvm_agent;
    `uvm_component_utils(fifo_agent);

    fifo_driver driver;
    fifo_sequencer sqr;
    fifo_monitor mon;
    fifo_config fifo_cfg;

    uvm_analysis_port #(fifo_seq_item) agt_ap;

```

```

function new(string name="fifo_agent", uvm_component parent = null);
    super.new(name, parent);
endfunction

function void build_phase (uvm_phase phase);
    super.build_phase(phase);

    driver = fifo_driver::type_id::create("fifo_driver", this);
    sqr = fifo_sequencer::type_id::create("fifo_sequencer", this);
    mon = fifo_monitor::type_id::create("fifo_monitor", this);
    fifo_cfg = fifo_config::type_id::create("fifo_config");
    agt_ap = new("agt_ap", this);
    if(!uvm_config_db #(fifo_config) :: get(this, "", "CFG", fifo_cfg))begin
        `uvm_fatal("build_phase", "Unable to get configuration object")
    end
endfunction

function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    driver.FIFO_vif = fifo_cfg.FIFO_vif;
    mon.FIFO_vif = fifo_cfg.FIFO_vif;
    driver.seq_item_port.connect(sqr.seq_item_export);
    mon.mon_ap.connect(agt_ap);
endfunction

endclass

endpackage

```

interface:

```

interface FIFO_if(clk);
    parameter FIFO_WIDTH = 16;
    parameter FIFO_DEPTH = 8 ;
    input bit clk ;
    logic [FIFO_WIDTH-1:0] data_in;
    logic rst_n, wr_en, rd_en;
    logic [FIFO_WIDTH-1:0] data_out;
    logic wr_ack, overflow;
    logic full, empty, almostfull, almostempty, underflow;

```

```

modport DUT (
input clk ,rst_n,wr_en,rd_en,data_in,
output data_out , wr_ack,overflow, full , empty, almostfull, almostempty,
underflow
);
modport TEST (
input clk ,data_out,wr_ack,overflow,full,empty,almostfull,almostempty,underflow,
output rst_n, wr_en , rd_en,data_in
);
modport MONITOR (input clk, rst_n, wr_en, rd_en , data_in, data_out, wr_ack,
overflow, full, empty, almostfull, almostempty, underflow);
endinterface

```

sva:

```

module FIFO_sva (FIFO_if fifoif);

parameter FIFO_DEPTH = 8;

// RESET CHECK

property reset_p;
  @(posedge fifoif.clk)
  !fifoif.rst_n | =>
    (fifoif.wr_ack == 0 &&
     fifoif.overflow == 0 &&
     fifoif.underflow == 0);
endproperty

reset_ap: assert property (reset_p);
reset_cp: cover property (reset_p);

// WRITE ACK CHECK

property wr_ack_p;
  @(posedge fifoif.clk) disable iff (!fifoif.rst_n)
  (!fifoif.full && fifoif.wr_en) | => fifoif.wr_ack;
endproperty

wr_ack_ap: assert property (wr_ack_p);

```

```

wr_ack_cp: cover property (wr_ack_p);

// OVERFLOW CHECK

property overflow_p;
  @(posedge fifoif.clk) disable iff (!fifoif.rst_n)
    (fifoif.full && fifoif.wr_en) | => fifoif.overflow;
endproperty

overflow_ap: assert property (overflow_p);
overflow_cp: cover property (overflow_p);

// UNDERFLOW CHECK

property underflow_p;
  @(posedge fifoif.clk) disable iff (!fifoif.rst_n)
    (fifoif.empty && fifoif.rd_en) | => fifoif.underflow;
endproperty

underflow_ap: assert property (underflow_p);
underflow_cp: cover property (underflow_p);

// FULL FLAG CHECK

property full_flag_p;
  @(posedge fifoif.clk) disable iff (!fifoif.rst_n)
    fifoif.full | -> fifoif.full;
endproperty

full_flag_ap: assert property (full_flag_p);

// EMPTY FLAG CHECK

property empty_flag_p;
  @(posedge fifoif.clk) disable iff (!fifoif.rst_n)
    fifoif.empty | -> fifoif.empty;
endproperty

```

```

empty_flag_ap: assert property (empty_flag_p);

// ALMOST FULL CHECK

property almostfull_p;
  @(posedge fifoif.clk) disable iff (!fifoif.rst_n)
    fifoif.almostfull |-> fifoif.almostfull;
endproperty

almostfull_ap: assert property (almostfull_p);

// ALMOST EMPTY CHECK

property almostempty_p;
  @(posedge fifoif.clk) disable iff (!fifoif.rst_n)
    fifoif.almostempty |-> fifoif.almostempty;
endproperty

almostempty_ap: assert property (almostempty_p);

// VALID READ DATA CHECK

property read_valid_p;
  @(posedge fifoif.clk) disable iff (!fifoif.rst_n)
    (fifoif.rd_en && !fifoif.empty) |=> !$isunknown(fifoif.data_out);
endproperty

read_valid_ap: assert property (read_valid_p);

// VALID WRITE CHECK

property write_valid_p;
  @(posedge fifoif.clk) disable iff (!fifoif.rst_n)
    (fifoif.wr_en && !fifoif.full) |=> fifoif.wr_ack;
endproperty

write_valid_ap: assert property (write_valid_p);

endmodule

```

rst_seq:

```
package fifo_reset_seq_pkg;

import fifo_seq_item_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"

class fifo_reset_seq extends uvm_sequence #(fifo_seq_item);
`uvm_object_utils(fifo_reset_seq);
    fifo_seq_item seq_item;

    function new(string name = "fifo_reset_seq");
        super.new(name);
    endfunction

    task body;
        seq_item = fifo_seq_item::type_id::create("seq_item");
        start_item(seq_item);
        seq_item.data_in = 0;
        seq_item.wr_ack = 0;
        seq_item.rd_en = 0;
        seq_item.rst_n = 0;
        finish_item(seq_item);
    endtask
endclass

endpackage
```

read_write_seq:

```
package fifo_write_read_seq_pkg;
import fifo_seq_item_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"

class fifo_write_read_seq extends uvm_sequence #(fifo_seq_item);
`uvm_object_utils(fifo_write_read_seq);
    fifo_seq_item seq_item;
```

```

function new(string name = "fifo_write_read_seq");
    super.new(name);
endfunction

task body;
    seq_item = fifo_seq_item::type_id::create("seq_item");
    repeat(5000)begin
        start_item(seq_item);
        assert(seq_item.randomize());
        finish_item(seq_item);
    end
endtask
endclass

endpackage

```

read_seq:

```

package fifo_read_seq_pkg;
import fifo_seq_item_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"

class fifo_read_seq extends uvm_sequence #(fifo_seq_item);
    `uvm_object_utils(fifo_read_seq);
    fifo_seq_item seq_item;

function new(string name = "fifo_read_seq");
    super.new(name);
endfunction

task body;
    seq_item = fifo_seq_item::type_id::create("seq_item");
    repeat(5000)begin
        start_item(seq_item);
        assert(seq_item.randomize());
        seq_item.wr_en = 0;
        seq_item.rd_en = 1;
        finish_item(seq_item);
    end
endtask
endclass

```



```
endpackage
```

write_seq:

```
package fifo_write_seq_pkg;
import fifo_seq_item_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"

class fifo_write_seq extends uvm_sequence #(fifo_seq_item);
`uvm_object_utils(fifo_write_seq);
fifo_seq_item seq_item;

function new(string name = "fifo_write_seq");
    super.new(name);
endfunction

task body;
    seq_item = fifo_seq_item::type_id::create("seq_item");
    repeat(5000)begin
        start_item(seq_item);
        assert(seq_item.randomize());
        seq_item.wr_en = 1;
        seq_item.rd_en = 0;
        finish_item(seq_item);
    end
endtask
endclass

endpackage
```

sequencer:

```
package fifo_sequencer_pkg;

import fifo_seq_item_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"

class fifo_sequencer extends uvm_sequencer #(fifo_seq_item);
```

```

`uvm_component_utils(fifo_sequencer);

function new(string name="fifo_sequencer", uvm_component parent = null);
    super.new(name, parent);
endfunction

endclass

endpackage

```

item:

```

package fifo_seq_item_pkg;

import uvm_pkg::*;
`include "uvm_macros.svh"
parameter FIFO_WIDTH = 16;
parameter FIFO_DEPTH = 8;

class fifo_seq_item extends uvm_sequence_item;
`uvm_object_utils(fifo_seq_item);

rand logic [FIFO_WIDTH-1:0] data_in;
rand logic rst_n, wr_en, rd_en;
logic [FIFO_WIDTH-1:0] data_out;
logic wr_ack, overflow;
logic full, empty, almostfull, almostempty, underflow;

int RD_EN_ON_DIST, WR_EN_ON_DIST;

function new (string name = "fifo_seq_item", int RD_EN_ON_DIST_params=30, int
WR_EN_ON_DIST_params=70);
    super.new(name);

    RD_EN_ON_DIST = RD_EN_ON_DIST_params;
    WR_EN_ON_DIST = WR_EN_ON_DIST_params;
endfunction

constraint Reset_con {
    rst_n dist {1:/90, 0:/10};
}

constraint Wr_en_con{

```

```

    wr_en dist {1:/WR_EN_ON_DIST, 0:/(100-WR_EN_ON_DIST)};
}

constraint Rd_en_con{
    wr_en dist {1:/RD_EN_ON_DIST, 0:/(100-RD_EN_ON_DIST)};
}

function string convert2string();
    return $sformatf("rst_n = %0d, wr_en = %0d, rd_en = %0d, data_in = %0d,
data_out = %0d, wr_ack = %0d, overflow = %0d, full = %0d, empty = %0d, almostfull
= %0d, almostempty = %0d, underflow = %0d",
                    rst_n, wr_en, rd_en, data_in, data_out, wr_ack, overflow,
full, empty, almostfull, almostempty, underflow);
endfunction

endclass

endpackage

```

scoreboard:

```

package fifo_scoreboard_pkg;

import uvm_pkg::*;
`include "uvm_macros.svh"
import fifo_seq_item_pkg::*;

class fifo_scoreboard extends uvm_scoreboard;

    `uvm_component_utils(fifo_scoreboard)

    uvm_analysis_export #(fifo_seq_item) sb_export;
    uvm_tlm_analysis_fifo #(fifo_seq_item) sb_fifo;

    fifo_seq_item tr;

    bit [FIFO_WIDTH-1:0] ref_data_out;
    bit ref_wr_ack, ref_overflow;
    bit ref_full, ref_empty, ref_almostfull, ref_almostempty, ref_underflow;

```

```

bit [FIFO_WIDTH-1:0] ref_mem [$];

int error_cnt = 0;
int correct_cnt = 0;

function new(string name="fifo_scoreboard", uvm_component parent=null);
    super.new(name,parent);
endfunction

function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    sb_export = new("sb_export", this);
    sb_fifo    = new("sb_fifo", this);
endfunction

function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    sb_export.connect(sb_fifo.analysis_export);
endfunction

task run_phase(uvm_phase phase);
    super.run_phase(phase);

    forever begin
        sb_fifo.get(tr);
        ref_model(tr);

        if(tr.data_out != ref_data_out) begin
            error_cnt++;
            `uvm_info("run_phase", $sformatf(
                "error %s , out_ref:%0h , wr_ack_ref:%0b, overflow_ref:%0b,
underflow_ref:%0b, full_ref:%0b, empty_ref:%0b, almostfull_ref:%0b,
almostempty_ref:%0b",
                tr.convert2string(), ref_data_out, ref_wr_ack, ref_overflow,
ref_underflow, ref_full, ref_empty, ref_almostfull, ref_almostempty
            ), UVM_MEDIUM);
        end
        else begin
            correct_cnt++;
        end
    end
end

```

```

        end
    endtask

    task ref_model(fifo_seq_item tr);

        if(!tr.rst_n) begin
            ref_wr_ack = 0;
            ref_overflow = 0;
            ref_underflow = 0;
            ref_full = 0;
            ref_empty = 1;
            ref_almostempty = 0;
            ref_almostfull = 0;
            ref_mem.delete();
        end
        else begin

            if(tr.wr_en && tr.rd_en && (ref_mem.size()==0 || ref_mem.size()==8))

begin

                if(ref_mem.size()==0) begin
                    ref_mem.push_back(tr.data_in);
                    ref_wr_ack = 1;
                    ref_overflow = 0;
                    ref_underflow = 0;
                end
                else if(ref_mem.size()==8) begin
                    ref_data_out = ref_mem.pop_front();
                    ref_underflow = 0;
                    ref_underflow = 0;
                end

            end

        end
        else begin

            if(tr.wr_en) begin
                ref_underflow = 0;
                if(ref_mem.size() < 8) begin
                    ref_mem.push_back(tr.data_in);
                    ref_wr_ack = 1;
                    ref_overflow = 0;
                end
            end
            else begin
                ref_wr_ack = 0;
            end
        end
    end
endtask

```

```

        ref_overflow = 1;
    end
end

    if(tr.rd_en) begin
        ref_overflow = 0;
        if(ref_mem.size() != 0) begin
            ref_data_out = ref_mem.pop_front();
            ref_underflow = 0;
        end
        else begin
            ref_underflow = 1;
        end
    end
end

end
end

if(ref_mem.size() == FIFO_DEPTH)
    ref_full = 1;
else
    ref_full = 0;

if(ref_mem.size() == 0)
    ref_empty = 1;
else
    ref_empty = 0;

if(ref_mem.size() == FIFO_DEPTH-1)
    ref_almostfull = 1;
else
    ref_almostfull = 0;

if(ref_mem.size() == 1)
    ref_almostempty = 1;
else
    ref_almostempty = 0;

endtask

function void report_phase(uvm_phase phase);

```

```

        super.report_phase(phase);

        `uvm_info("report_phase", $sformatf("correct: %0d", correct_cnt),
UVM_MEDIUM);
        `uvm_info("report_phase", $sformatf("error: %0d", error_cnt),
UVM_MEDIUM);
    endfunction

endclass

endpackage

```

monitor:

```

package fifo_monitor_pkg;
`include "uvm_macros.svh"
import fifo_seq_item_pkg::*;
import uvm_pkg::*;

class fifo_monitor extends uvm_monitor;
    `uvm_component_utils(fifo_monitor)

    virtual FIFO_if FIFO_vif;
    uvm_analysis_port #(fifo_seq_item) mon_ap;
    fifo_seq_item rsp_seq_item;

    function new(string name = "fifo_monitor", uvm_component parent = null);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        mon_ap = new("mon_ap", this);
    endfunction: build_phase

    task run_phase(uvm_phase phase);
        super.run_phase(phase);
        forever begin
            rsp_seq_item = fifo_seq_item::type_id::create("rsp_seq_item");
            @(negedge FIFO_vif.clk);
            rsp_seq_item.data_in = FIFO_vif.data_in;

```

```

        rsp_seq_item.rst_n = FIFO_vif.rst_n;
        rsp_seq_item.wr_en = FIFO_vif.wr_en;
        rsp_seq_item.rd_en = FIFO_vif.rd_en;
        rsp_seq_item.data_out = FIFO_vif.data_out;
        rsp_seq_item.wr_ack = FIFO_vif.wr_ack;
        rsp_seq_item.overflow = FIFO_vif.overflow;
        rsp_seq_item.full = FIFO_vif.full;
        rsp_seq_item.empty = FIFO_vif.empty;
        rsp_seq_item.almostfull = FIFO_vif.almostfull;
        rsp_seq_item.almostempty = FIFO_vif.almostempty;
        rsp_seq_item.underflow = FIFO_vif.underflow;
        mon_ap.write(rsp_seq_item);
        `uvm_info("run_phase", rsp_seq_item.convert2string(), UVM_HIGH)
    end
endtask

endclass
endpackage

```

top:

```

import uvm_pkg::*;
`include "uvm_macros.svh"
import fifo_test_pkg::*;

module top();

    bit clk;
    initial begin
        forever
            #1 clk = ~clk;
    end
    FIFO_if fifoif (clk);

    FIFO DUT (fifoif);

    bind FIFO FIFO_sva svas (.fifoif(fifoif));

    initial begin
        uvm_config_db #(virtual FIFO_if)::set (null, "uvm_test_top", "FIFO_if",
        fifoif);
        run_test("fifo_test");
    end

endmodule

```


test:

```
package fifo_test_pkg ;

import fifo_cfg_pkg::*;
import fifo_env_pkg::*;
import uvm_pkg::*;
import fifo_write_seq_pkg::*;
import fifo_read_seq_pkg::*;
import fifo_write_read_seq_pkg::*;
import fifo_reset_seq_pkg::*;
`include "uvm_macros.svh"

class fifo_test extends uvm_test;
`uvm_component_utils(fifo_test)

fifo_env env;
fifo_config fifo_cfg;
fifo_reset_seq reset_seq;
fifo_write_seq write_seq;
fifo_read_seq read_seq;
fifo_write_read_seq write_read_seq;
virtual FIFO_if FIFO_vif;
function new (string name = "fifo_test", uvm_component parent = null);
    super.new(name, parent);
endfunction

function void build_phase(uvm_phase phase);
    super.build_phase(phase);

    env = fifo_env::type_id::create("env", this);
    fifo_cfg = fifo_config::type_id::create("fifo_cfg");
    reset_seq = fifo_reset_seq::type_id::create("fifo_reset_seq");
    write_seq = fifo_write_seq::type_id::create("fifo_write_seq");
    read_seq = fifo_read_seq::type_id::create("fifo_read_seq");
    write_read_seq = fifo_write_read_seq::type_id::create("fifo_write_read_seq");

    if(!uvm_config_db #(virtual FIFO_if)::get(this, "", "FIFO_if",
fifo_cfg.FIFO_vif))
        `uvm_fatal("build phase", "Cannot get the virtual interface");
endfunction
endclass
```

```

    uvm_config_db #(fifo_config) :: set (this, "*", "CFG", fifo_cfg);
endfunction

task run_phase(uvm_phase phase);
    super.run_phase(phase);

    phase.raise_objection(this);
    `uvm_info("run_phase", "reset_seq Asserted", UVM_MEDIUM)
    reset_seq.start(env.agt.sqr);
    `uvm_info("run_phase", "reset_seq deAsserted", UVM_MEDIUM)

    `uvm_info("run_phase", "write_seq Asserted", UVM_MEDIUM)
    write_seq.start(env.agt.sqr);
    `uvm_info("run_phase", "write_seq deAsserted", UVM_MEDIUM)

    `uvm_info("run_phase", "read_seq Asserted", UVM_MEDIUM)
    read_seq.start(env.agt.sqr);
    `uvm_info("run_phase", "read_seq deAsserted", UVM_MEDIUM)

    `uvm_info("run_phase", "write_read_seq Asserted", UVM_MEDIUM)
    write_read_seq.start(env.agt.sqr);
    `uvm_info("run_phase", "write_read_seq deAsserted", UVM_MEDIUM)

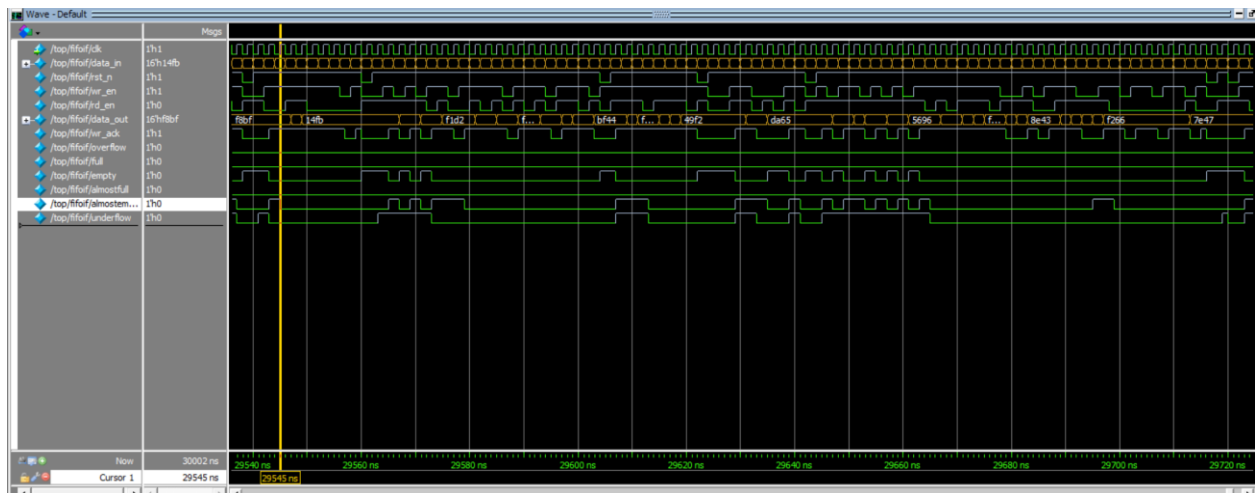
    phase.drop_objection(this);
endtask

endclass

endpackage

```

questa:



Name	Assertion Type	Language	Enable	Failure Count	Pass Count	Active Count	Memory	Peak Memory	Peak Memory Time	Cumulative Threads	ATV	Assertion Expression	Included
▲ /uvm_pkg::uvm_re...	Immediate	SVA	on	0	0	-	-	-	-	-	off	assert (\$cast(seq.o))	✗
▲ /uvm_pkg::uvm_re...	Immediate	SVA	on	0	0	-	-	-	-	-	off	assert (\$cast(seq.o))	✗
▲ /fifo_write_read_s...	Immediate	SVA	on	0	1	-	-	-	-	-	off	assert (randomize(...))	✓
▲ /fifo_read_seq_pk...	Immediate	SVA	on	0	1	-	-	-	-	-	off	assert (randomize(...))	✓
▲ /fifo_write_seq_pk...	Immediate	SVA	on	0	1	-	-	-	-	-	off	assert (randomize(...))	✓
▲ /top/DUT/svas/res...	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert (@(posedge ffof.clk) (~ffoi...	✓
▲ /top/DUT/svas/rw...	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert (@(posedge ffof.clk) disabi...	✓
▲ /top/DUT/svas/ove...	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert (@(posedge ffof.clk) disabi...	✓
▲ /top/DUT/svas/und...	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert (@(posedge ffof.clk) disabi...	✓
▲ /top/DUT/svas/full...	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert (@(posedge ffof.clk) disabi...	✓
▲ /top/DUT/svas/em...	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert (@(posedge ffof.clk) disabi...	✓
▲ /top/DUT/svas/alm...	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert (@(posedge ffof.clk) disabi...	✓
▲ /top/DUT/svas/alm...	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert (@(posedge ffof.clk) disabi...	✓
▲ /top/DUT/svas/rea...	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert (@(posedge ffof.clk) disabi...	✓
▲ /top/DUT/svas/writ...	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert (@(posedge ffof.clk) disabi...	✓

Name	Language	Enabled	Log	Count	AtLeast	Limit	Weight	Cmplt %	Cmplt graph	Included	Memory	Peak Memory	Peak Memory Time	Cumulative Threads
▲ /top/DUT/svas/res...	SVA	✓	Off	1518	1	Unli...	1	100%	✓	✓	0	0	0 ns	0
▲ /top/DUT/svas/rw...	SVA	✓	Off	4276	1	Unli...	1	100%	✓	✓	0	0	0 ns	0
▲ /top/DUT/svas/ove...	SVA	✓	Off	1791	1	Unli...	1	100%	✓	✓	0	0	0 ns	0
▲ /top/DUT/svas/und...	SVA	✓	Off	4648	1	Unli...	1	100%	✓	✓	0	0	0 ns	0

```

# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO : 14
# UVM_WARNING : 0
# UVM_ERROR : 0
# UVM_FATAL : 0
# ** Report counts by id
# [Questa UVM] 2
# [RNTST] 1
# [TEST_DONE] 1
# [report_phase] 2
# [run_phase] 8
# ** Note: $finish : D:/questasim64_2021.1/win64/./verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)

```